Search Structures and Algorithms

ITCS 4114: Real World Algorithms

Kalpathi Subramanian

Computer Science, UNC Charlotte

Last Modified: March, 2026

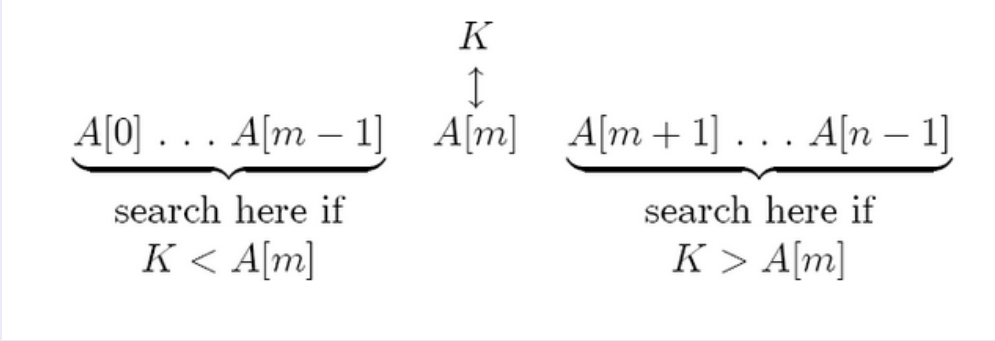
Search Algorithms

- Searching for information has been estimated to nearly **60% or more** of what computing systems do
- With the internet, search engines and AI based search, the estimate is likely **conservative.**
- Efficient search algorithms and data structures are needed for retrieving information from very **large data collections.**
- We will look at both **1D and 2D** (spatial) data structures for searching, as well as structures for efficient search of databases.

One Dimensional Search

Binary Search on Sorted Lists

- Uses a decrease and conquer strategy
- At each step, the mid point value is compared to search key and the search moves to the left or right partition, decreasing the search space by half.



Complexity

$$T(n) = T(n/2) + 1$$

which by Master Theorem evaluates to

$$T(n) \in O(\lg n)$$

Binary Search

Algorithm

```
int Binary_Search(A[0...,N-1], K) {
    // Searches for key K on a sorted array A[0..,N-1]
    // Output: index of the key K or -1 if not found
    left = 0; right = N-1;
    while (l <= r) {
        m = (l+r)/2
        if (K == A[m])   // found it
            return m;
        else if (K < A[m])   // search left partition
            r = m-1;
        else                 // search right partition
            l = m+1;
    }
    return -1;               // not found
}
```

Problem: Binary Search

Assume your sorted list is stored in a singly linked list

- What is the complexity of searching for an item in the list?

$$T(n) = ?$$

- Verify using Master Theorem.
- Is it different using a doubly linked list?

Search Structures: Types

- Search trees are commonly used, as they (almost) guarantee $O(\lg N)$ worst case complexity
- Types of Tree structures:
  - **Binary search trees** are commonly used for search data that can be held in main memory
  - **Balanced search trees** (AVL, Red-Black, 2-3 trees) **guarantee** $O(\lg N)$ performance
  - **B-trees** are optimized for searching databases (seondary storage)
  - **Spatial Search Structures**, such as quadtrees and octrees are used to search geometric spaces, in such applications as computer graphics, robotics, game-based environments, etc.

Search Data Structures

Motivation

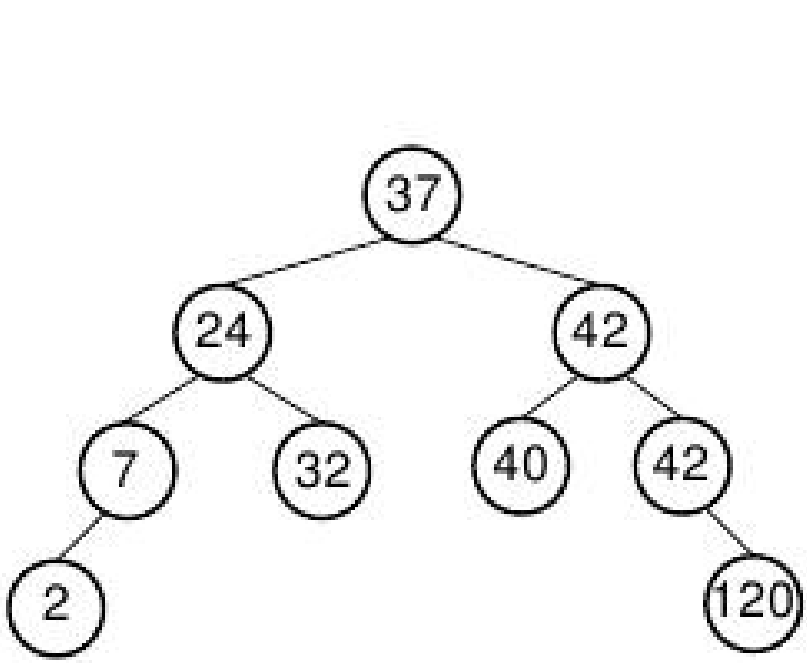
- Searching for a particular record in an unordered list takes $O(n)$, **too slow for large lists**
- If the list is ordered, can use an array implementation and use binary search; however, **insertion is** still $O(n)$.
- Is there a solution that allows **efficient search, insertion, deletion**?

Binary Search Trees

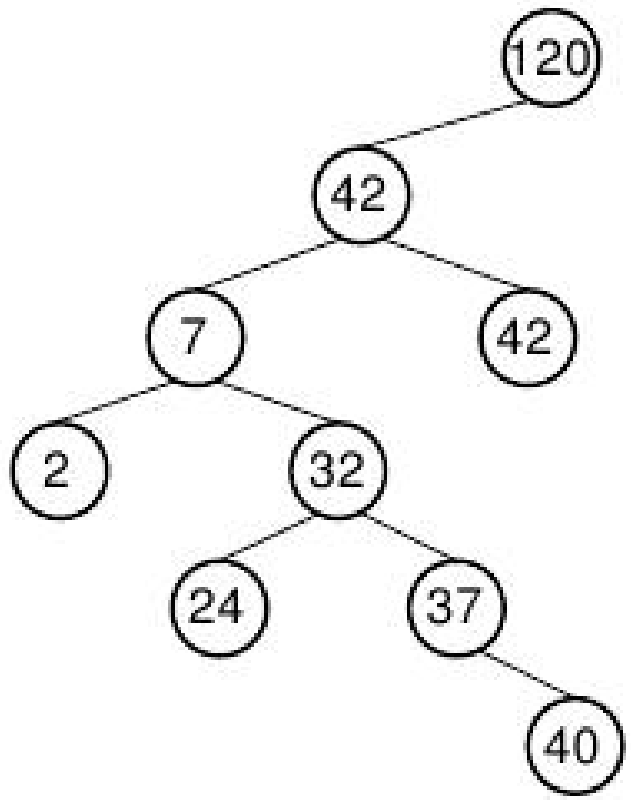
- **An efficient data structure** that can perform insertions, deletions and search in logarithmic time.
- Each node of the tree has a unique key.
- All keys in the left subtree of node with value K have values less than K
- All keys in the right subtree of node with value K have values greater than or equal to K

Binary Search Tree Examples

Binary search trees can be balanced or unbalanced



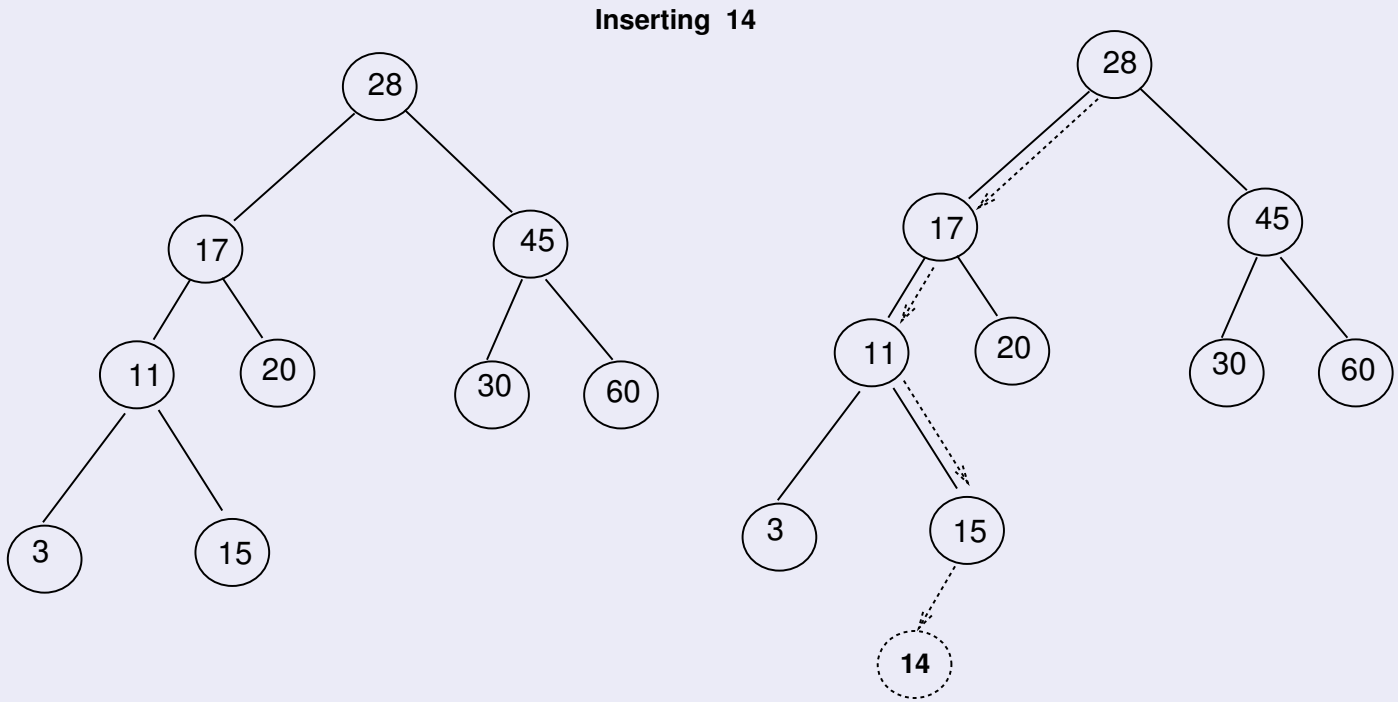
(a)



(b)

Binary Search Tree Operations: Insert

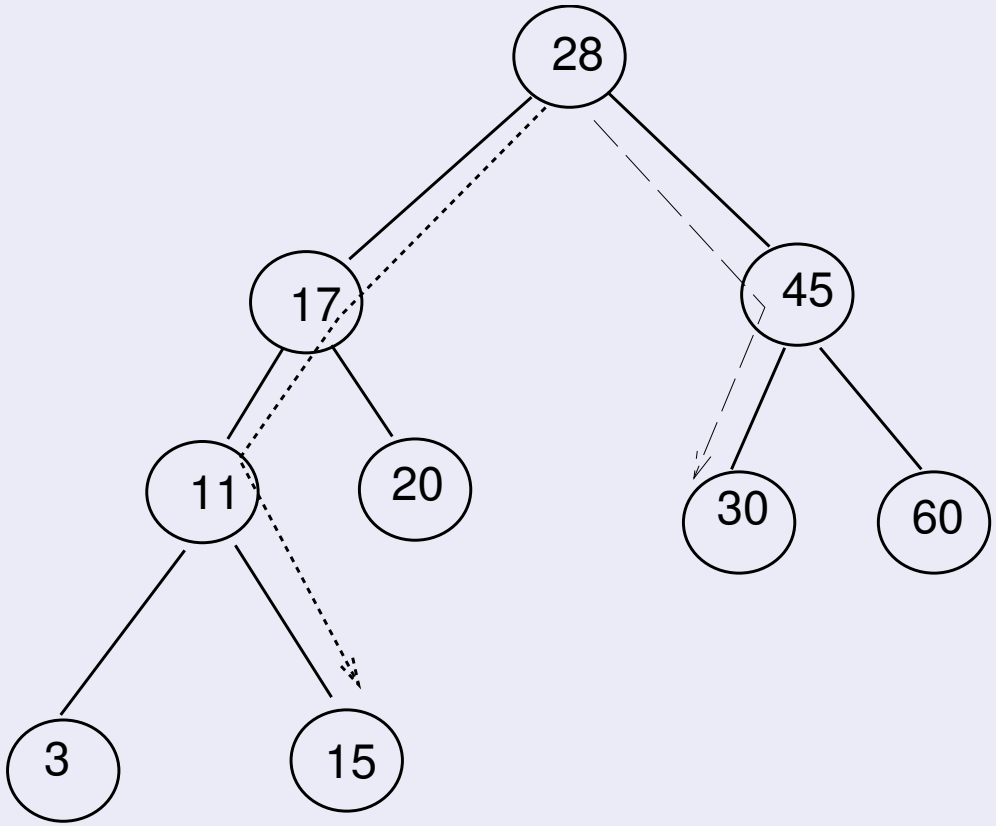
Insertion always creates a leaf node by following a path in the tree to find its locatrion



Binary Search Tree Operations: Find

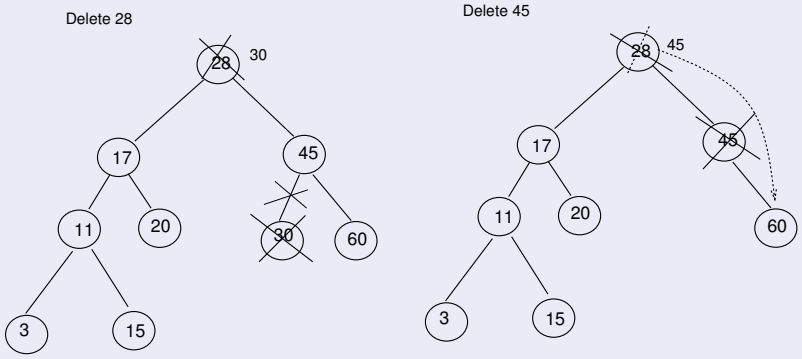
Finding a record is similar to insert, following a path to search for the key

Find 15, 30..



Binary Search Tree Operations: Removing a Node

More complicated, must consider cases of leaf node, nodes with 1 or 2 children.



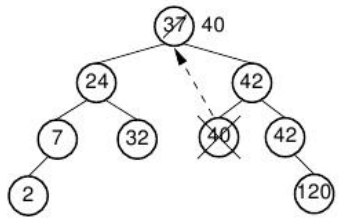


Figure 5.17 An example of removing the value 37 from the BST. The node containing this value has two children. We replace value 37 with the least value from the node's right subtree, in this case 40.

- If element is a leaf node, its parent becomes a node with null left or right subtree.
- If element's left subtree is NULL, make its parent point to its right subtree.
- If element's right subtree is NULL, make its parent point to its left subtree.
- Element with 2 children
  - Replace node with the largest node in left subtree, or,
  - Replace node with the smallest node in right subtree
  - Might want to choose based on the tree imbalance

Binary Search Tree Operations: Range Search

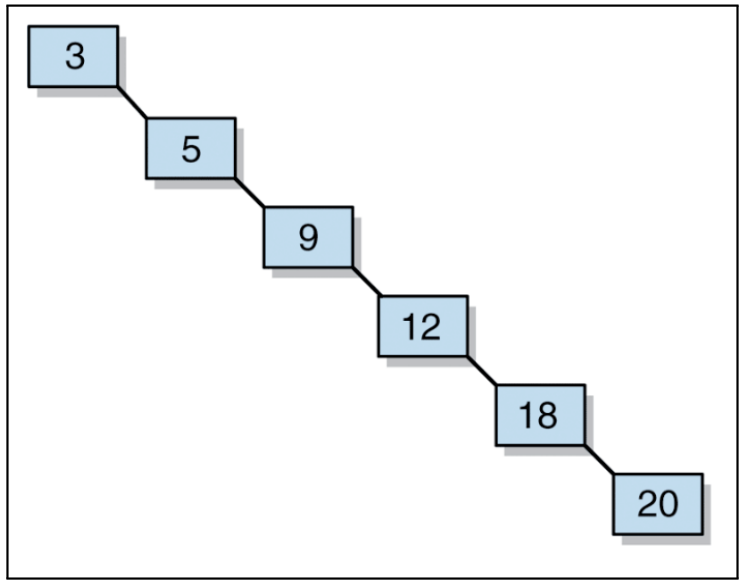
Identifying a range of keys

- Similar to insertion algorithm, but a bit tricky!
- Range (low to high) is input to the algorithm

Algorithm

```
RangeSearch(Tree root, int low, int high) {
if (root.key < low) {
    // can have values in range in right subtree
    // recurse on right subtree
}
else if (root.key > high)
    // can have values in range in left subtree
    // recurse on left subtree
else {
    // element in range, output element
    // recurse on both subtrees
}
```

Balanced Binary Search Trees



- Binary search trees lose their logarithmic complexity when they are unbalanced after a series of insert/delete operations
- Might need to rebalance periodically, which can be expensive
- **Example:** Consider inserting the elements in order: *3, 5, 9, 12, 18, 20*
- Tree operations complexity reduces to $O(n)$
- **Solution:** Tree structures that **continually maintain balance** with some additional work **without compromising computational complexity**.
- **Examples:** AVL trees, Red-Black trees

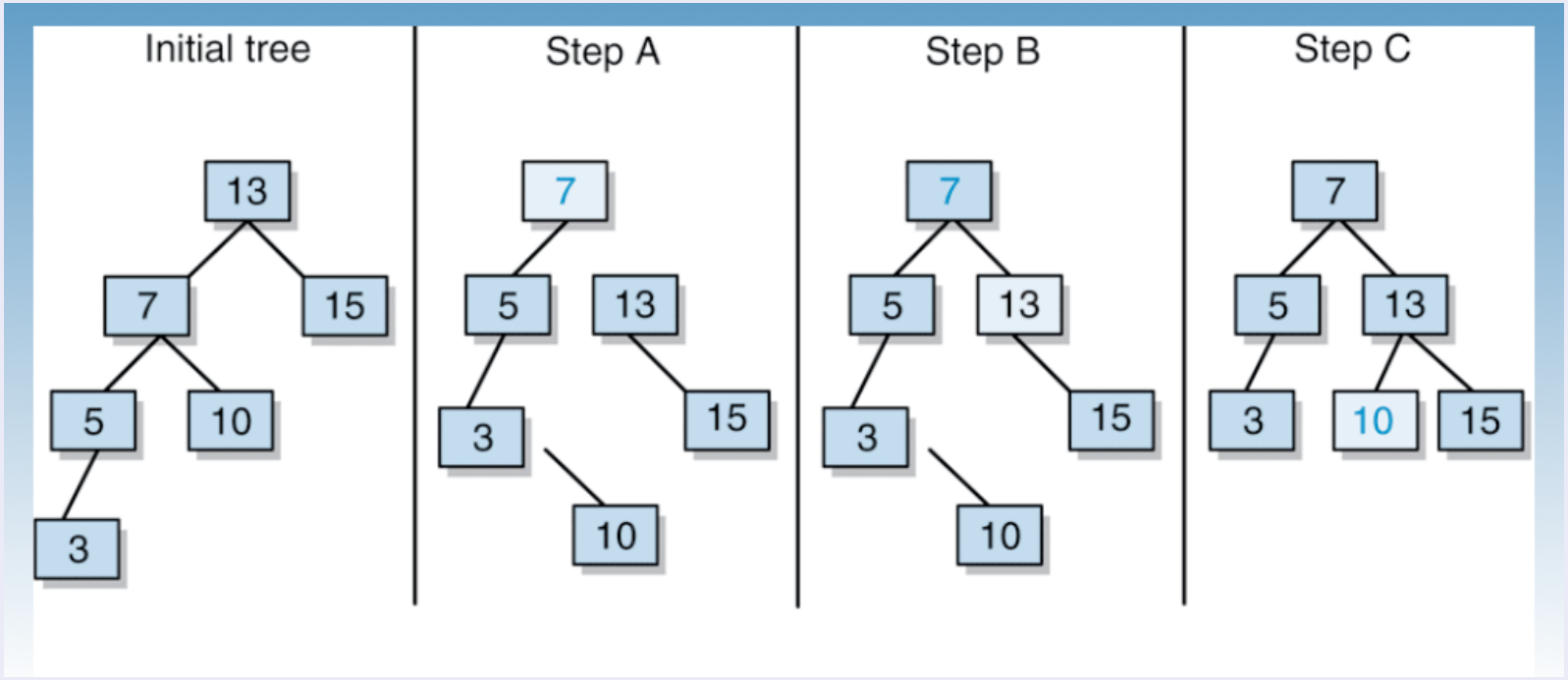
Balanced Search Trees

Two approaches:

- Keep tree **continually balanced**, by checking balance after each insert/delete operations and performing single and double **tree rotations** (AVL trees, Red-Black trees)
- Trees that are **always perfectly balanced through structure manipulation** – by growing the tree upwards towards the root (2-3 trees)

Tree Rotations: Right Rotation

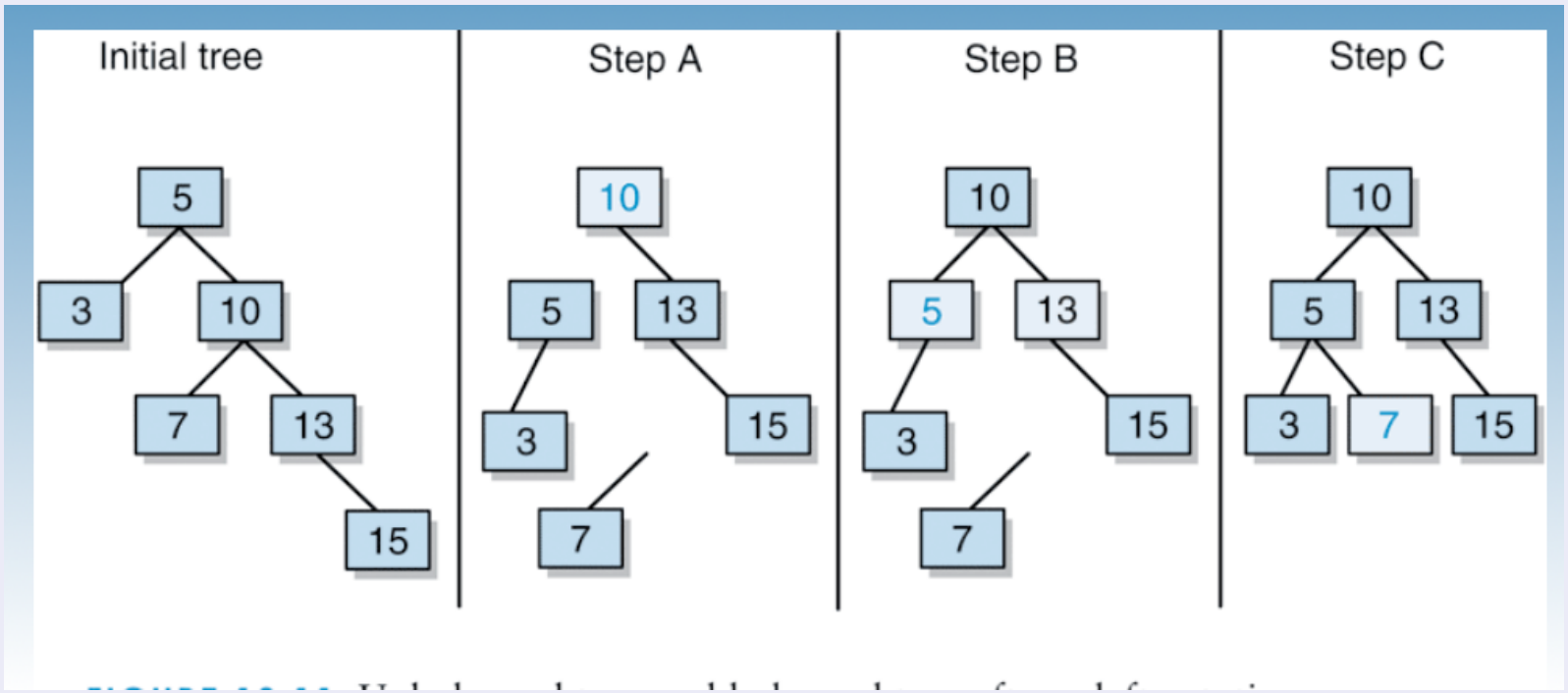
Caused by a long path in the left subtree of the left child of the root



- Imbalance detected at the node N (13)
- Rotate right: Left child of N comes up, N becomes right child of the new root (7) and N's right subtree becomes the left subtree of the old root.

Tree Rotations: Left Rotation

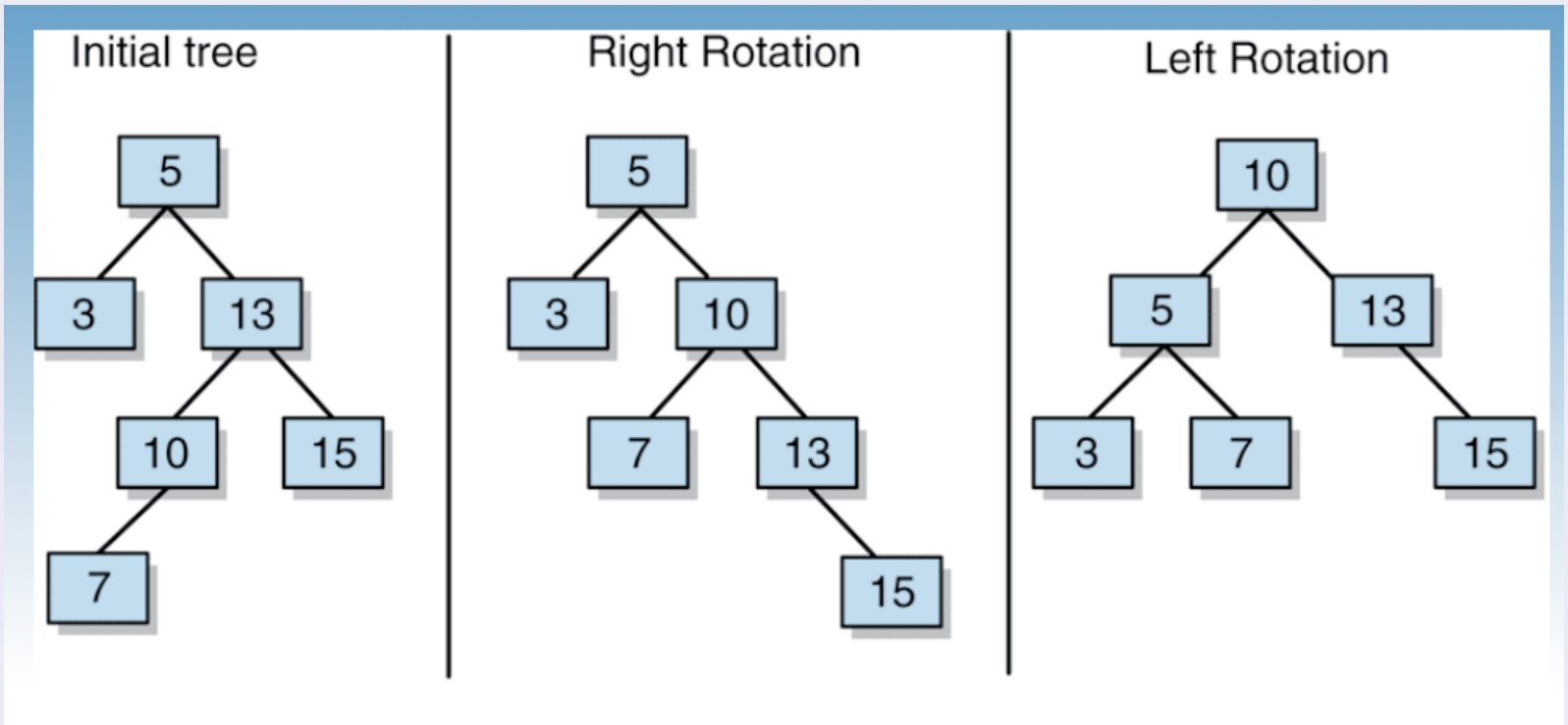
Caused by a long path in the right subtree of the right child of the root



- Imbalance detected at the node N (5)
- Rotate left: Right child (10) of N comes up, N becomes left child of the new root (10) and N's left subtree becomes the right subtree of the old root.

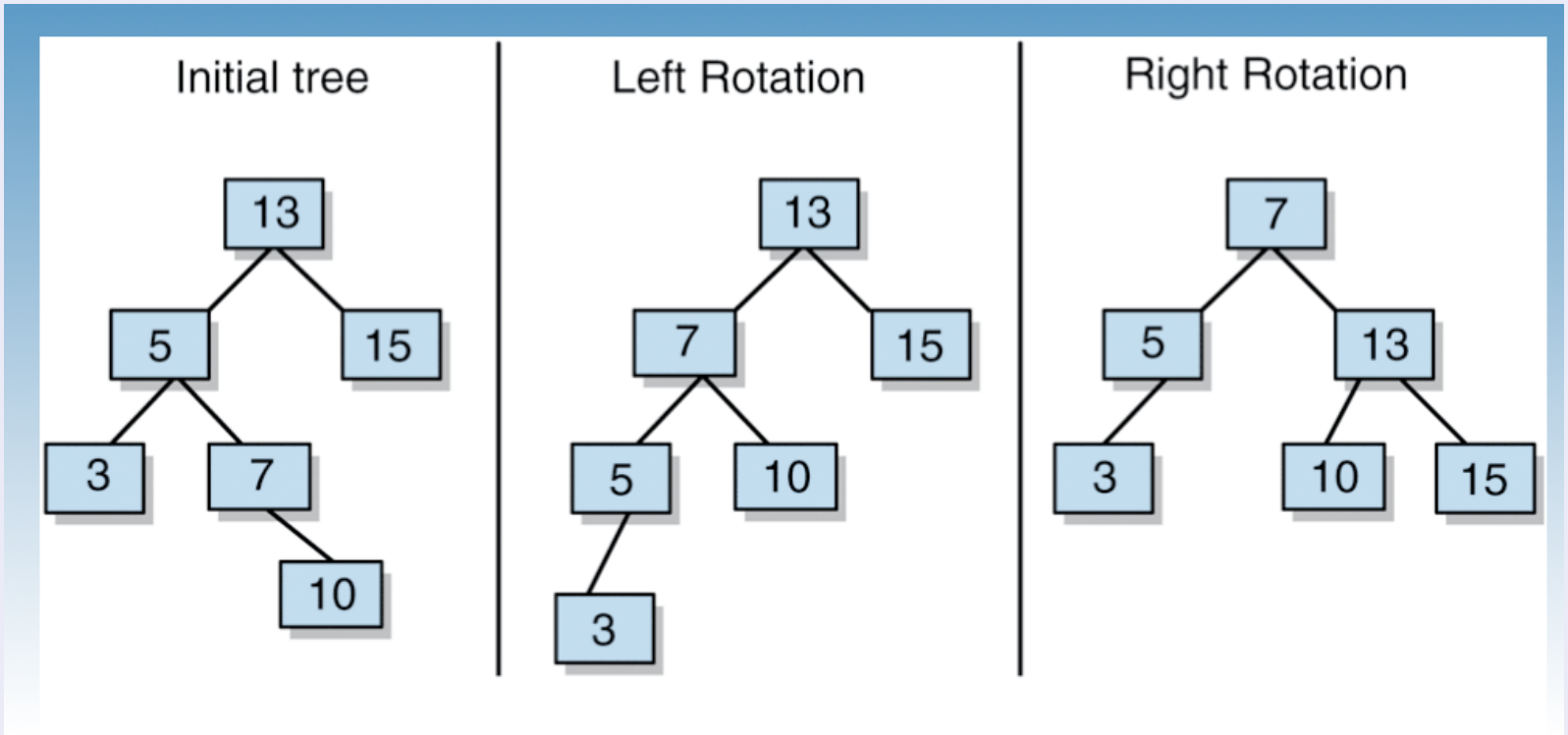
Tree Rotations: Right-Left Rotation

Solves an imbalance due to a long path in the left subtree of the right child of the root



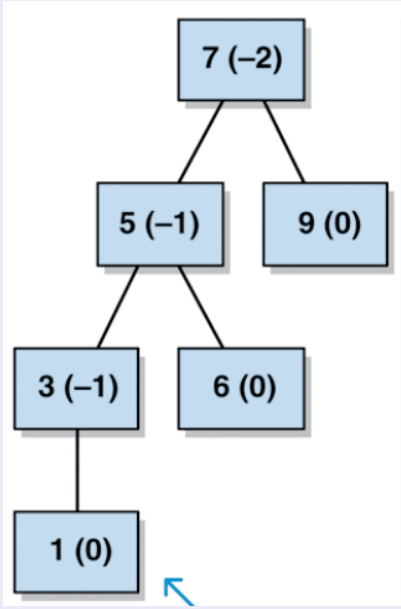
Tree Rotations: Left-Right Rotation

Solves an imbalance due to a long path in the right subtree of the left child of the root



AVL Trees

AVL trees keep track of the difference in height between the right and left sub-trees for each node, known as the "balance factor"

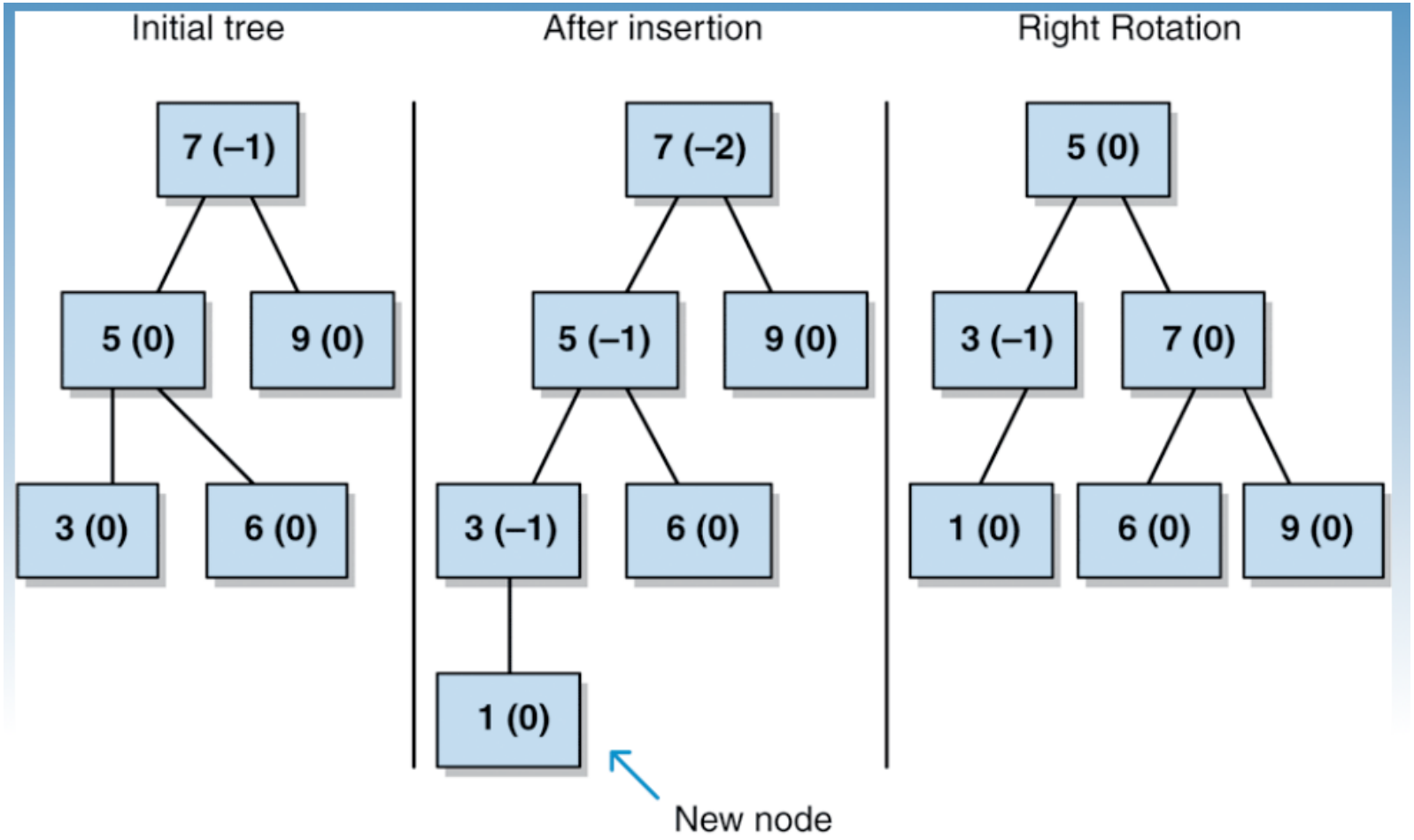


- If the balance factor of any node is less than -1 or greater than 1, then that sub-tree needs to be rebalanced using rotations.
- The balance factor of any node can only be changed through either insertion or deletion of nodes in the tree
- **Convention Used Here:** Balance factor $= Height_{right} - Height_{left}$

AVL Trees: Balancing Cases

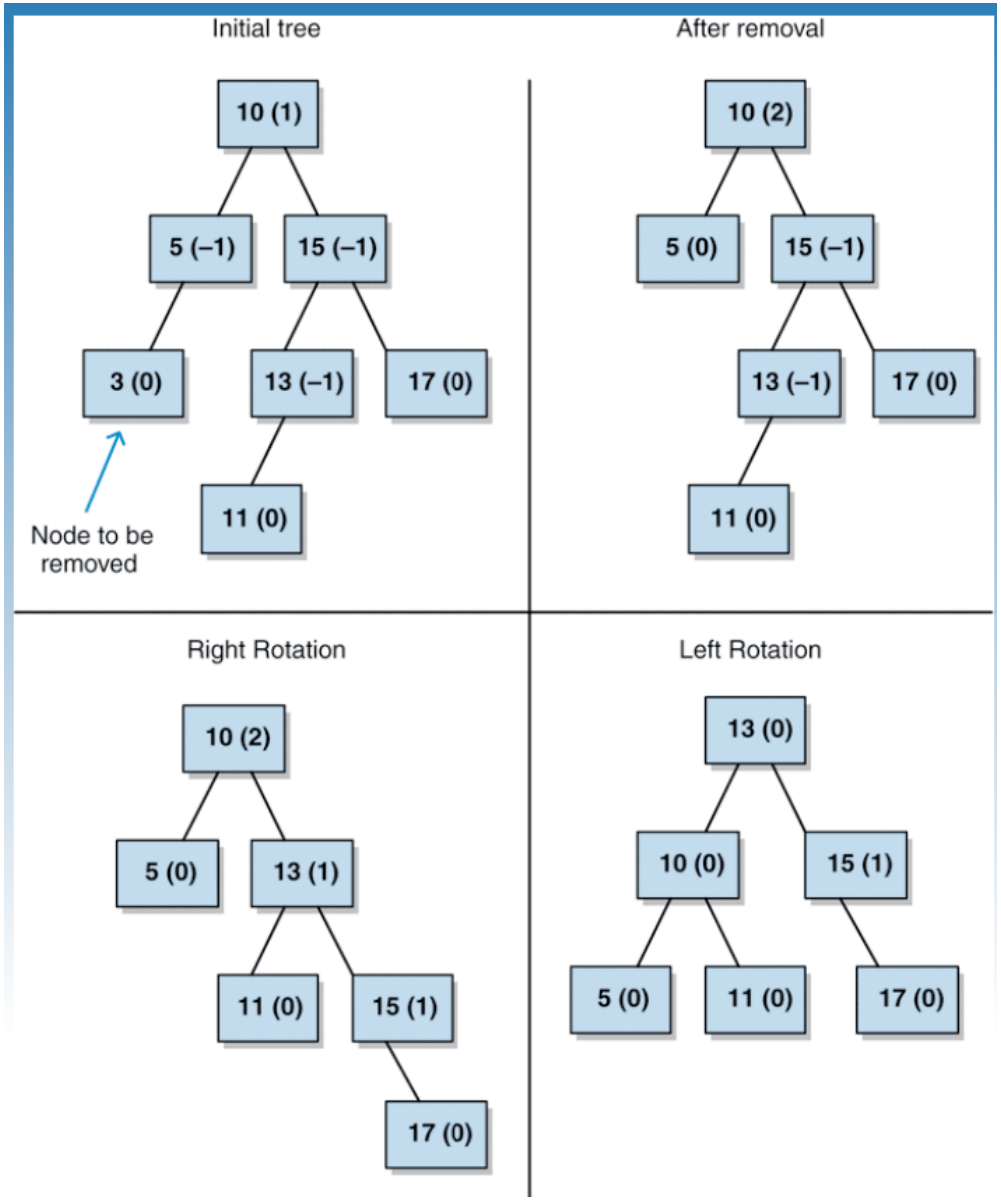
- **Right Rotation:** The balance factor at a node is -2 and its left child's balance factor is -1
- **Left Rotation:** The balance factor at a node is $+2$ and its right child's balance factor is $+1$
- **Right-Left Rotation:** Balance factor of a node is $+2$ and its right child's balance factor is -1 (long path in the left subtree of the node's right child)
- **Left-Right Rotation:** Balance factor of a node is -2 and its left child's balance factor is $+1$ (long path in the right subtree of the node's left child)

AVL Rotation Example: Right Rotation



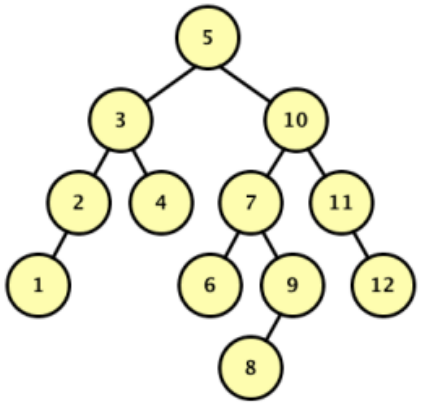
Note: Balance factors are computed as height of right tree - height of left tree

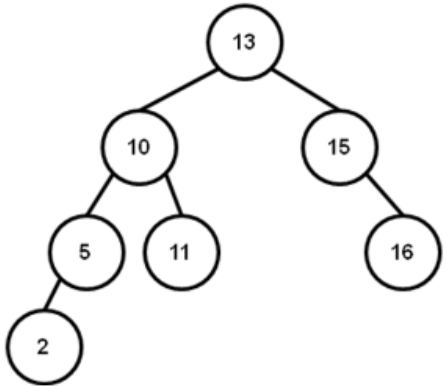
AVL Rotation Example: Right-Left Rotation

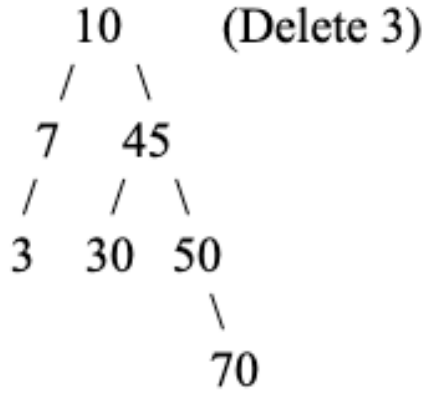


Note: Balance factors are computed as height of right tree - height of left tree

AVL Tree Exercises

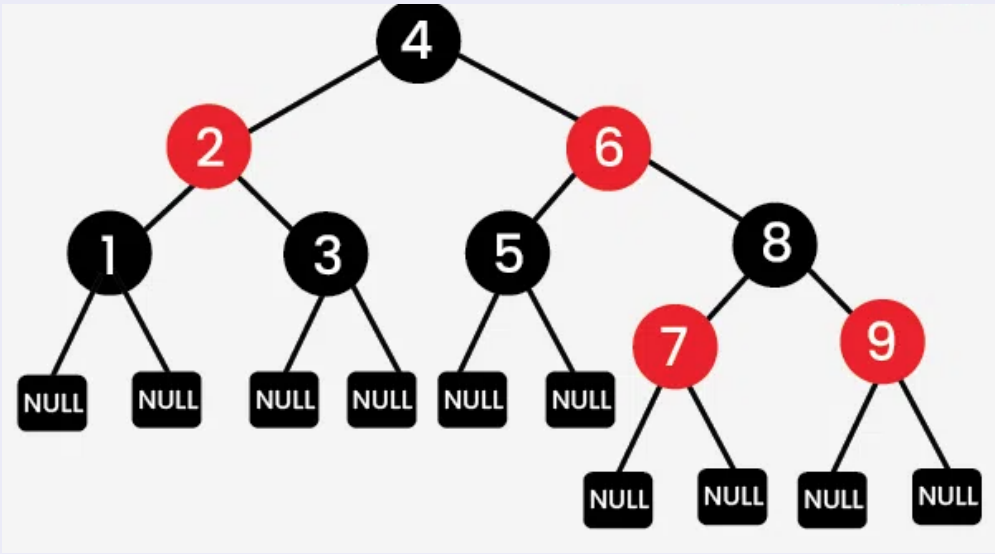






- **Left Tree:** Delete node 5 (use a node from the left subtree) and rebalance the AVL tree:
- **Middle Tree:** Insert 4 followed by 6 into the AVL tree below and rebalance as needed:
- **Right Tree:** Delete 3 and rebalance
- Insert the following keys into an initially empty AVL tree:
  21, 26, 30, 9, 4, 14, 28, 15, 10, 2, 3, 7

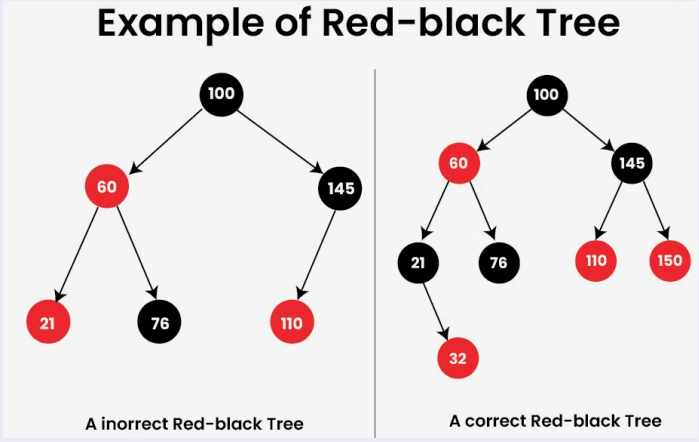
Red-Black Trees



- Red-Black trees are similar to AVL trees, can be thought as a commercial version of AVL trees
- Logarithmic complexity for insertion, deletion, find operations
- Involves rotations to build a consistent red-black tree
- Uses a **set of rules** to maintain balance
- For more info: *[Link] Intro to Red-Black Trees*

Red-Black Trees

Maintaining Balance: Rules



- Each node is either Red or Black
- Root node is always black
- **Red Property:** Red nodes cannot have red children (no two consecutive red nodes)
- **Black Property:** Every path from a node to its descendant null nodes has the same number of black nodes
- All null nodes (nodes with null children) are black.

During insertions/deletions, violations of these rules are resolved by rotations of respective nodes causing the violation

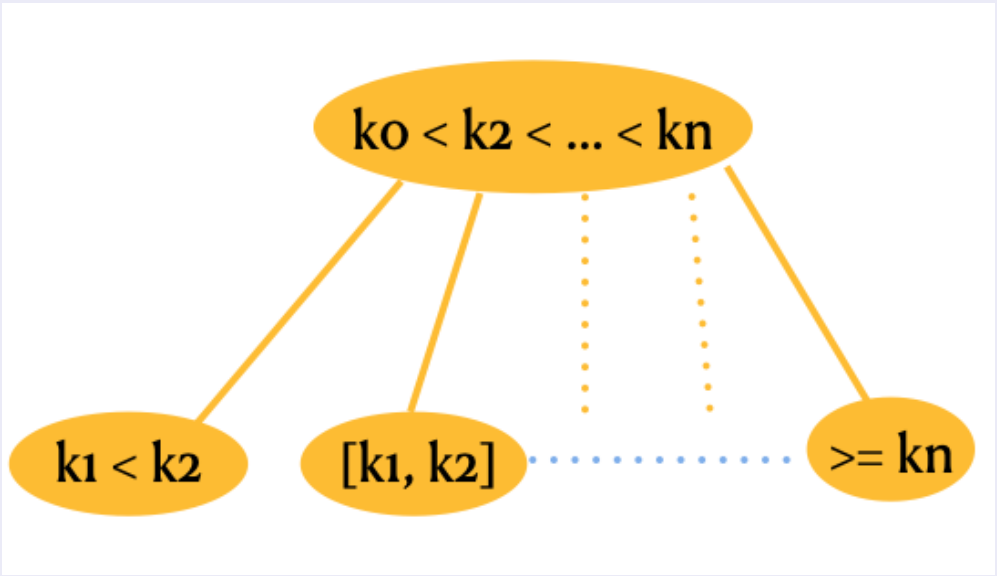
Red-Black and AVL Trees

- AVL Trees are more balanced compared to Red-Black trees, but might cause more rotations to occur during insertion/deletion
- AVL trees have stricter rules to maintain balance, hence faster searches, while Red-Black trees have looser balancing rules, leading to faster insertions/deletions
- Red-Black trees are preferred for frequent insertion/deletions, while AVL trees are better for more frequent search operations.
- Red-Black trees are more widely used, popular in implementation of maps, sets, priority queues (language standards typically dont dictate implementation)

Multiway Search Trees

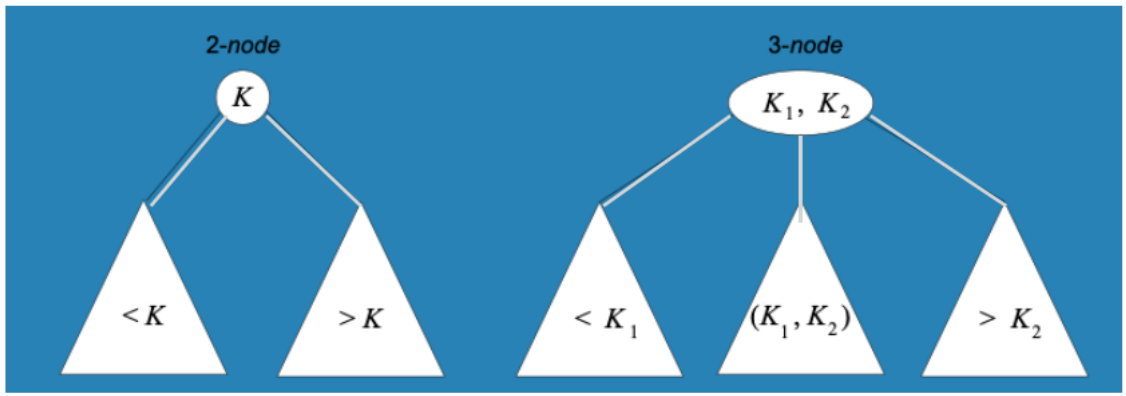
A search tree that allows more than one key in the same node of the tree

- A node of a search tree is **n-node** if it contains n-1 ordered keys and divides the range into **n intervals**, pointed to by the node's **n links** to their children
- A binary tree is a 2-node tree.



2-3 Trees

A binary search tree with 2-nodes and 3-nodes



- Similar to binary trees, new keys are inserted into the leaf of a tree
- If the leaf node is a 3-node, its split into two and middle key promoted to its parent

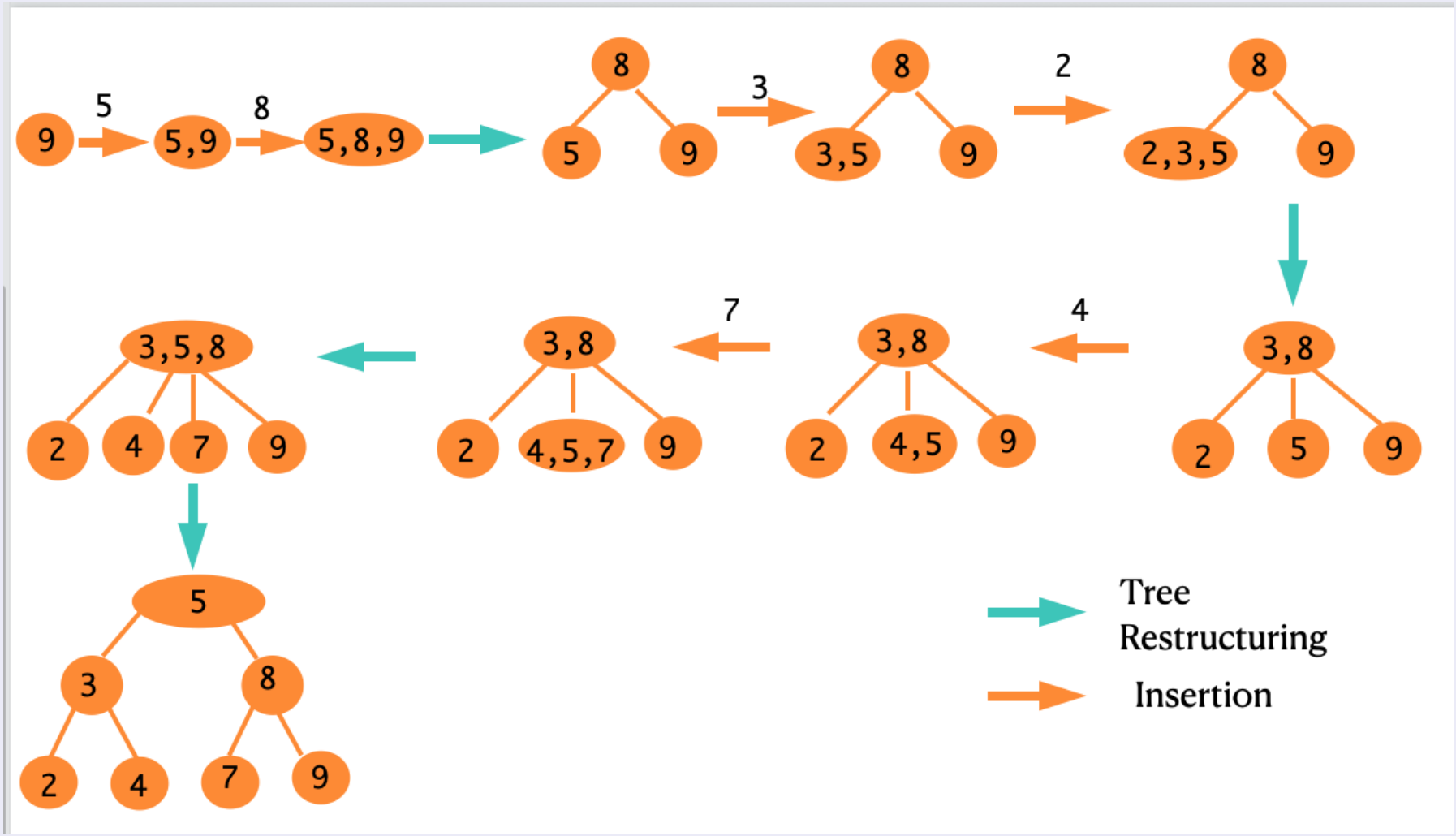
2-3 Tree Insertion

4 cases to consider:

- **Insert into a 2-node:** Replace 2 node with a 3-node
- **Insert into a tree with a single 3-node:** Add key to node, making a 4-node; promote middle key to a new root, resulting a 2-node tree
- **Insert into a 3-node whose parent is a 2-node:** Add key to node, making a 4-node; promote middle key to its parent, making a 3-node parent; split 4-node into two 2-nodes
- **Insert into a 3-node whose parent is a 3-node:** This is similar to the case 3, except that it can cause multiple keys to be promoted to their parents
- **Splitting the root:** Case 3 can reach the tree root, in which case the root is split (similar to case 3) and this **increases the height of the tree by 1**

2-3 Tree Construction Example

Inserting 9, 5, 8, 3, 2, 4, 7



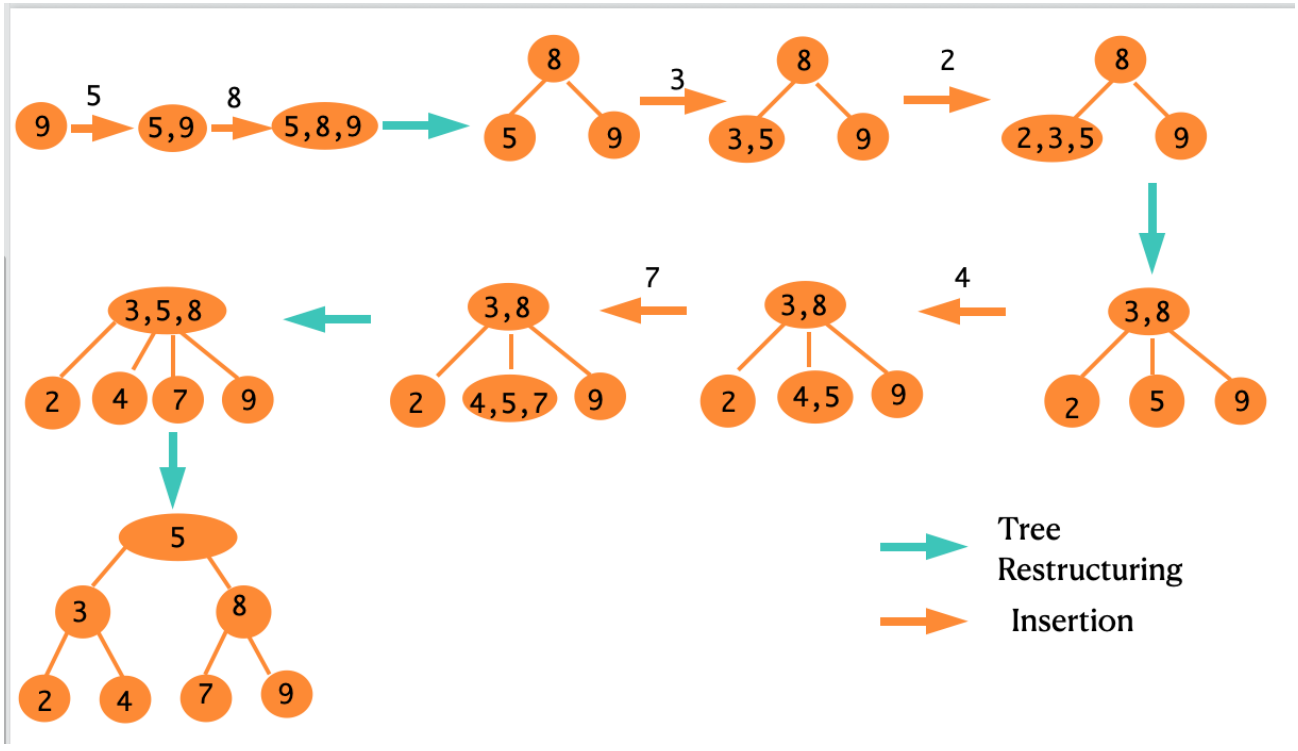
Analysis

- All operations on a 2-3 tree are guaranteed to visit at most $(\lg n + 1)$ nodes
- The height of the 2-3 tree is between $\lfloor \lg_3 n \rfloor$ and $\lfloor \lg_2 n \rfloor$

$$\lg_3(n+1) - 1 \leq h \leq \lg_2(n+1) - 1$$

  where h is the tree height, bound between full 2 node and 3 node trees.
- Work done at each node is a constant and operations (find, insert) examine a single path in the tree
- All operations are thus, $O(\lg n)$

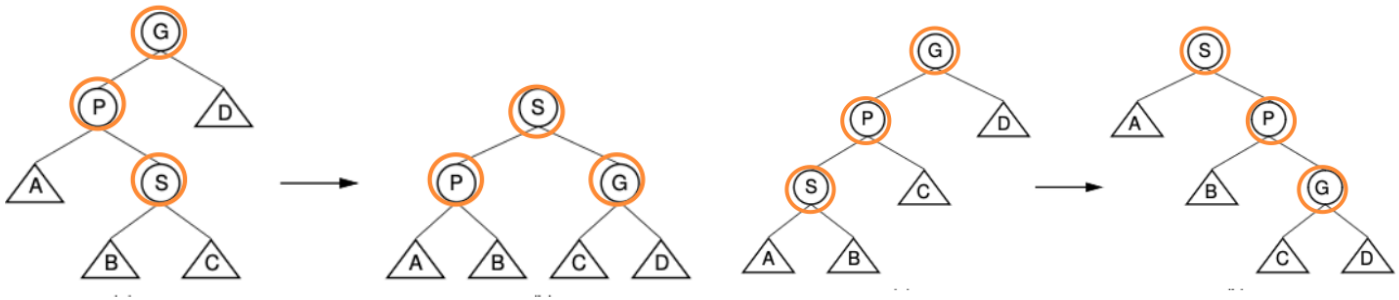
2-3 Tree Construction Exercises



- Build a 2-3 tree with the keys: 1,2,3,4,5,6,7,8,9
- Build a 2-3 tree with the keys: 6, 5, 4, 3, 2, 1
- Build a 2-3 tree with the keys: 3,6, 5, 1, 2, 4
- Build a 2-3 tree with the character keys: A, L, G, O, R, I, T, H, M
- Build a 2-3 tree with the character keys: C, O, M, P, U, T, I, N, G

Splay Tree

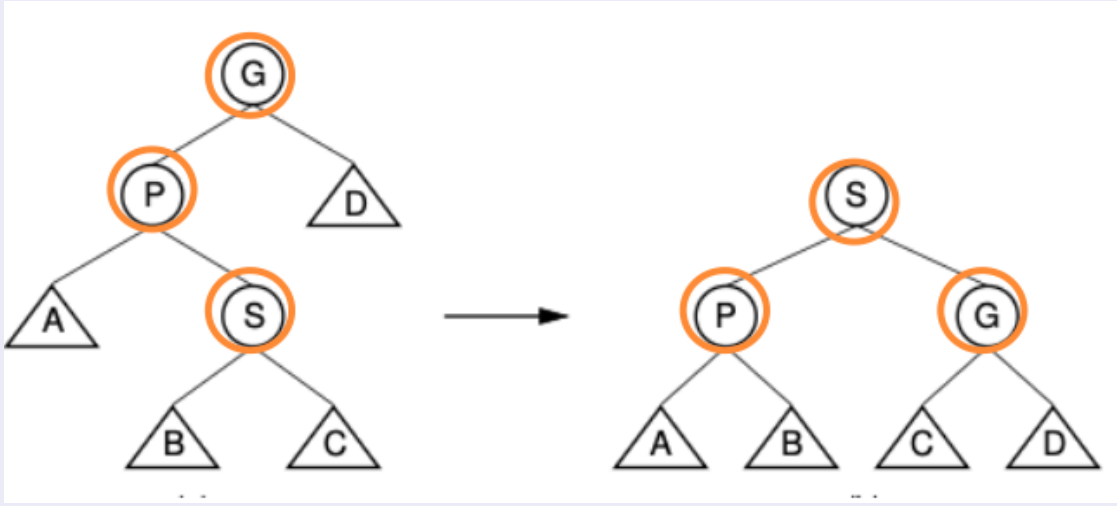
Similar to a binary search tree, but **guarantees complexity of** $O(m \lg n)$ **for** m **operations on tree of size** n



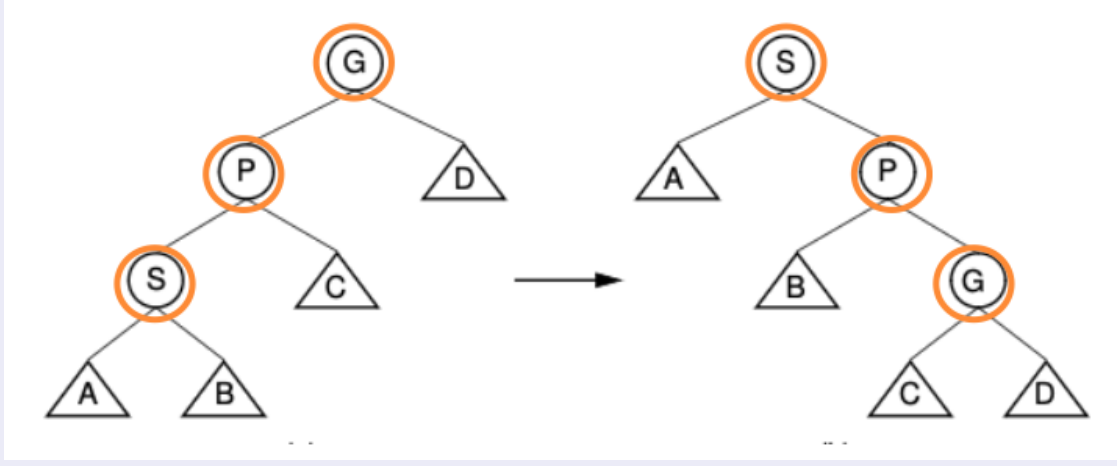
- Any individual operation can be $O(n)$, but m operations ($m > n$) will have an **average** complexity of $O(\lg n)$
- **Splaying:** Each access to a node moves it towards the root, leading to a more balanced tree, implemented through a series of **rotations**
- Rotation is performed to move the accessed node (can be through insert, delete, search), its parent and grand parent towards the root.
- **Zig (single)** or **Zig-Zag, Zig-Zig (double)** rotations

Splay Tree Rotations

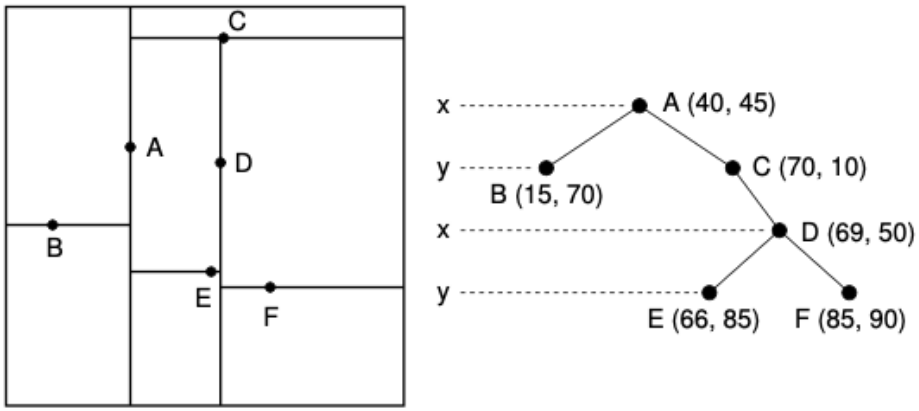
Zig-Zag: Subtrees S, P, and G altered to maintain BST property



Zig-Zig: Subtrees S, P, and G altered to maintain BST property



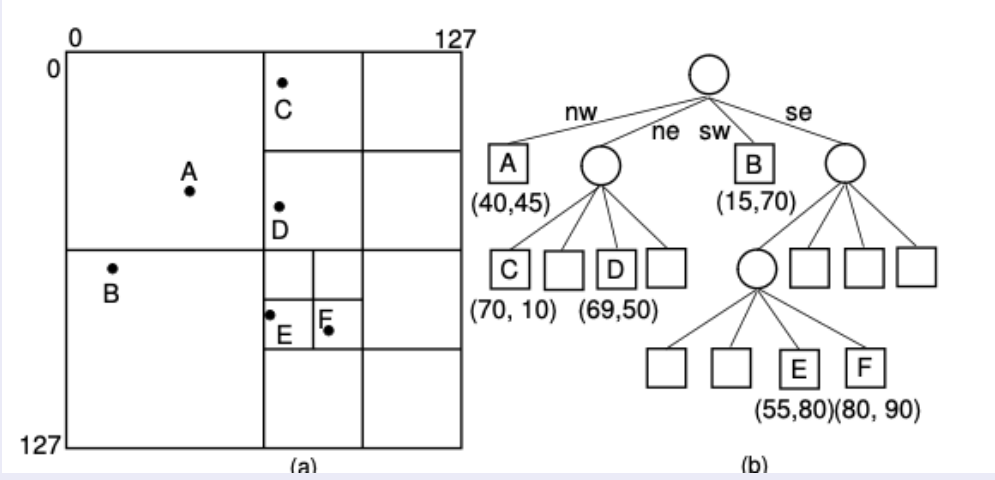
Spatial Data Structures



- So far we have seen tree structures for efficient search in **1 dimension**, with typically an integer (or any orderable type) key.
- Can be thought of as dividing a 1 dimensional space (line) into multiple segments
- One-dimensional queries do not suport range queries, such as finding all cities within a Lat/Long range.
- Multi-dimensional range queries are central to **spatial applications**
- Search space is now defined by the underlying **coordinate system** and search keys now represent **positions**
- **Applications:** Geographic Information Systems (GIS), Computer Graphics, Robotics

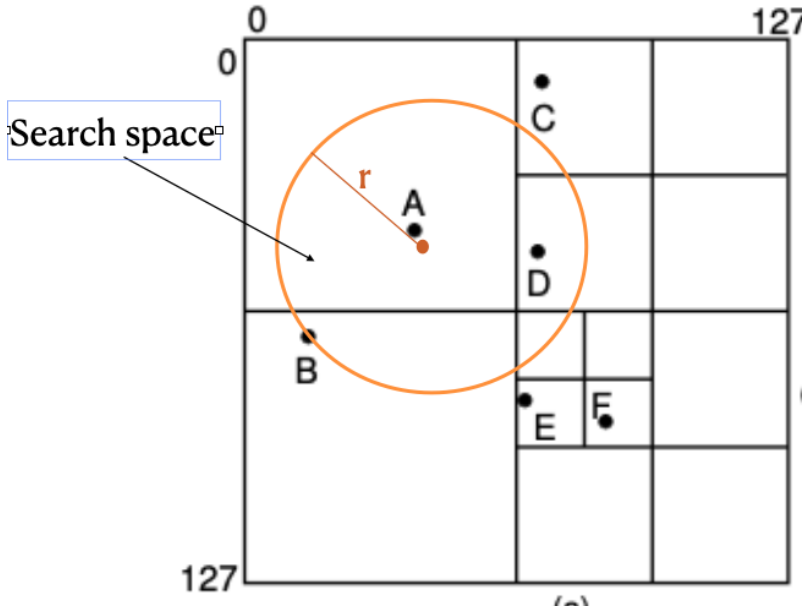
Spatial Data Structures

Point-Region (PR) Quadtree



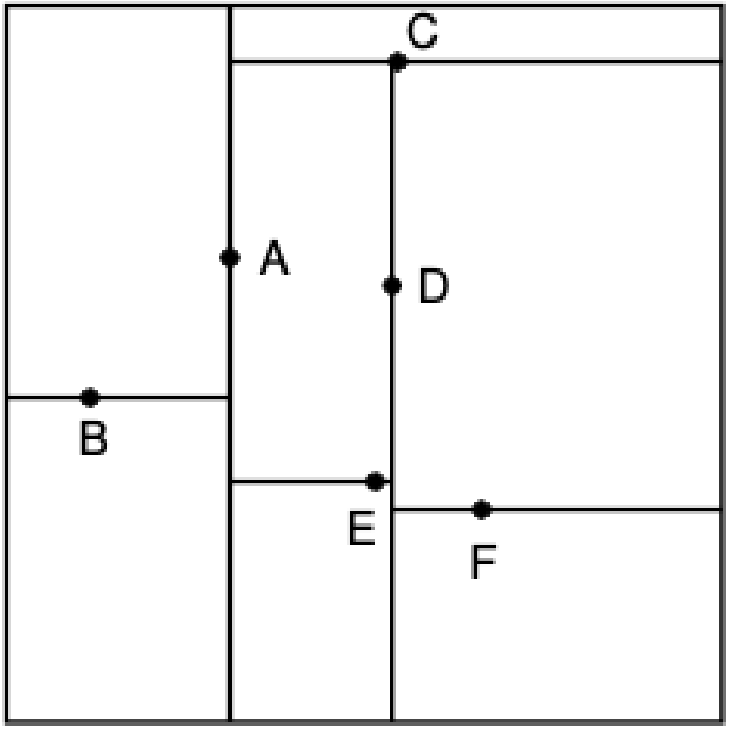
- An **adaptive search structure** that **recursively** subdivides space into 4 equal sized regions
- Representation for points/objects that adapt to its **density** in space
- Searching for an item is similar to a BST, except that each node represents 4 regions or a single terminating region (leaf node)
- Insertion into a quadtree can cause **new subdivisions** of space, deletions can cause **collapses** of space.
- Applications include **computer graphics, vision, robotics**

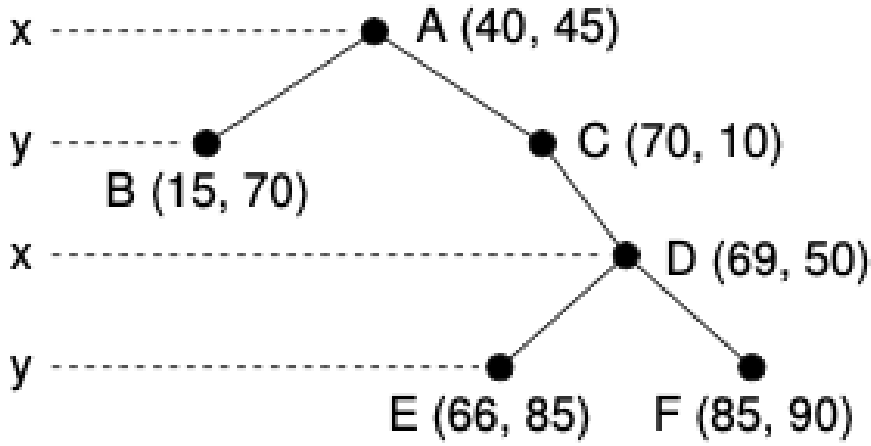
PR Quadtree: Operations



- **Insertion:** Similar to a BST, use the 2D key to look for the leaf node containing the point - may result in subdividing the region
- **Deletion:** Search for the point; if found, remove it - might result in collapsing the region (if the 4 siblings become empty) and, possibly its parent(s).
- **Range Search:** Can specify radial search region (as shown above) and look for points in quadrants that intersect the region.

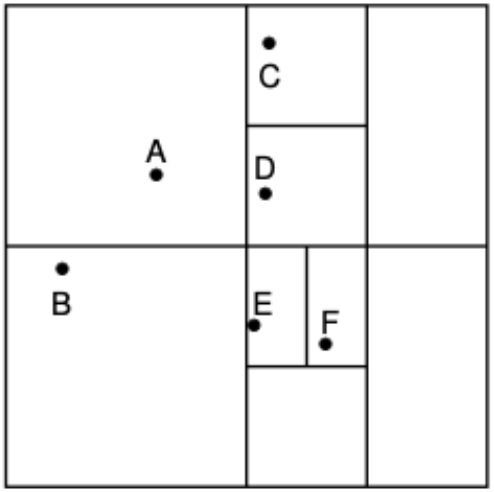
K-D Tree

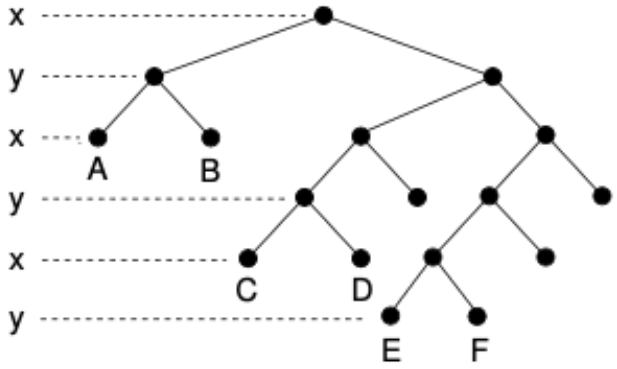


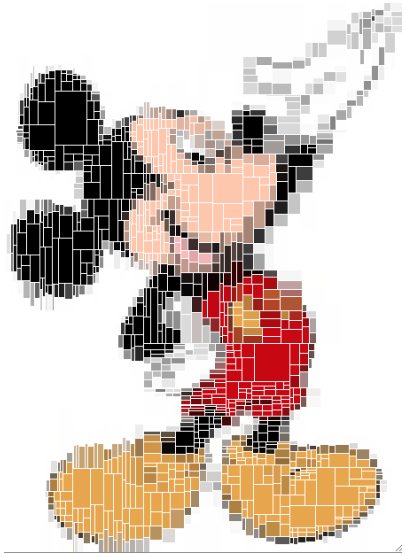


- K represents the dimension
- Partitions are based on the key, also termed the **discriminator**
- Similar to a binary search tree, but handles multiple keys
- Tree is constructed using a rotation of the keys
- Predominantly used a spatial search structure
- Partitions can be flexible (dont have to pass through points/objects)

Bintree







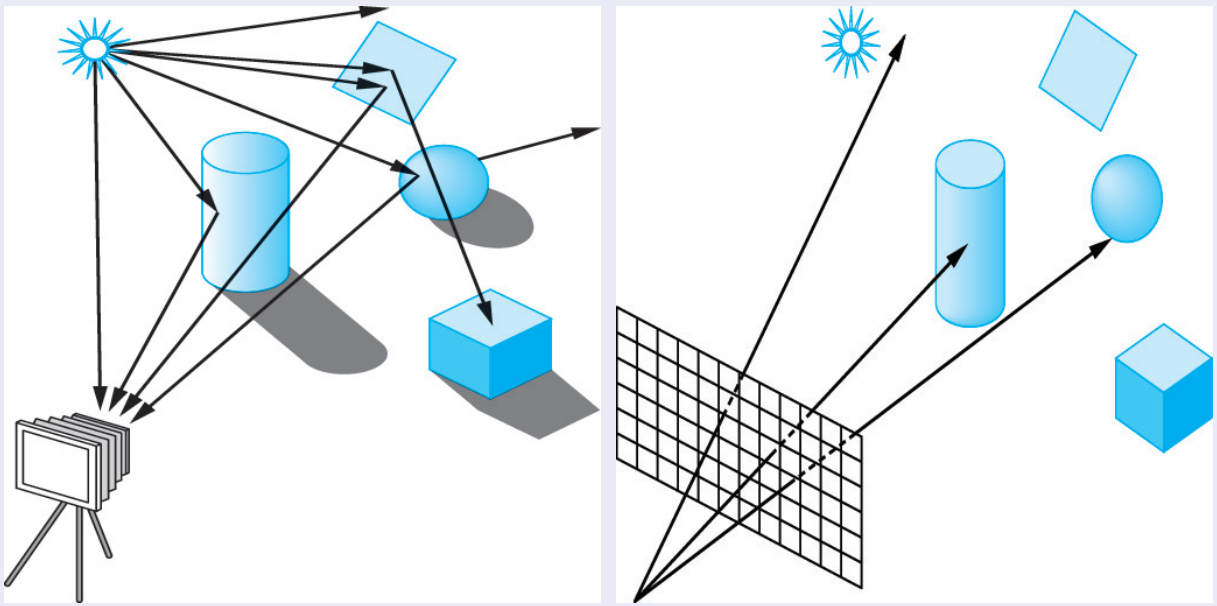
- Essentially a K-d tree, but subdivides multi-dimensional space
- Still a binary tree, rotating the subdivision through each dimension (X, Y, Z, X, Y, Z, X...)

Spatial Data Structures

Application:Computer Graphics: Ray Tracing Algorithms for Realistic Image Generation

- Generate realistic projected 2D images of 3D environments taking into account light interactions with objects in the environment.
- Minimize computation using spatial data structures.

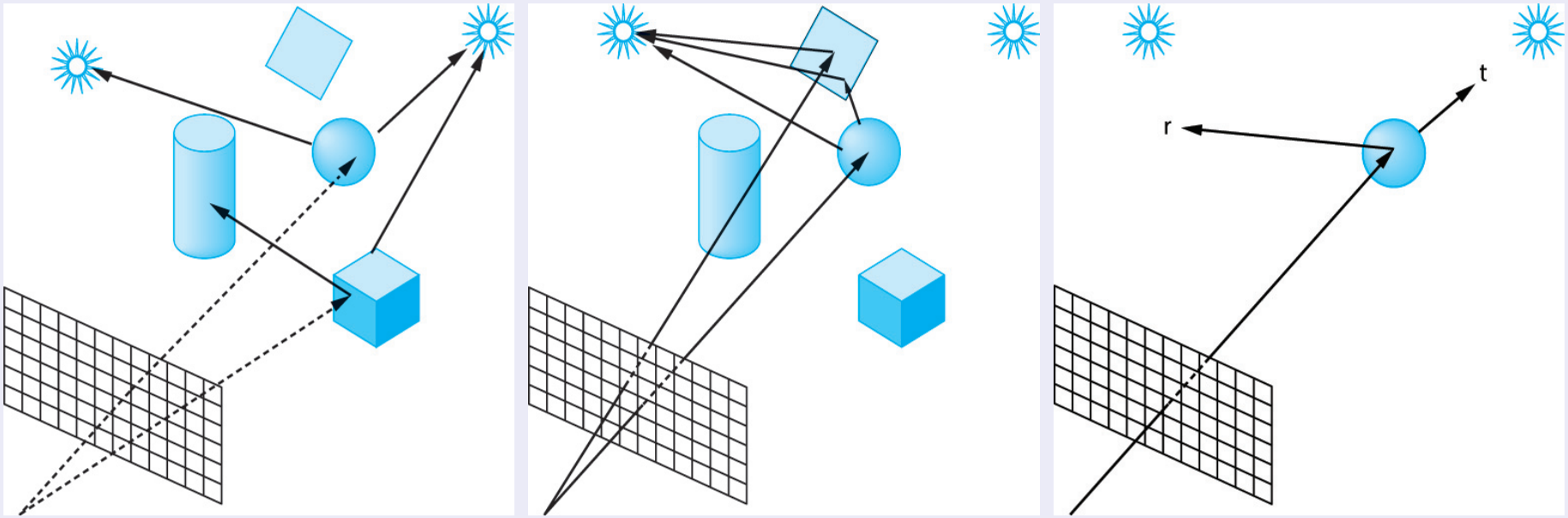
Problem Description



Courtesy:Angel/Shreiner (Figs. 12.1, 12.2)

Ray Tracing Algorithm

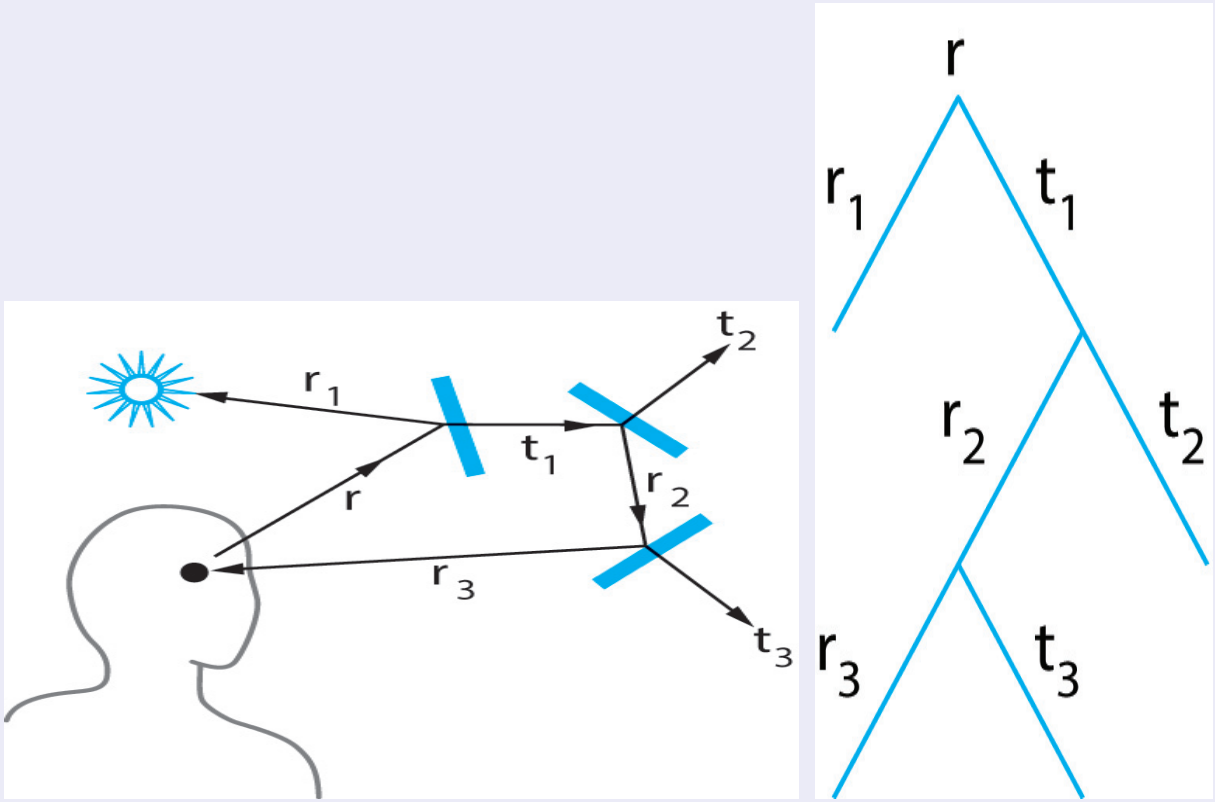
Algorithm simulates light interaction (reflection, refraction) taking into account material properties, visibility, with repeated interactions (scattering



Courtesy:Angel/Shreiner (Figs. 12.3, 12.4, 12.5)

Ray Tracing Algorithm

Simple Ray Tracing Model



Courtesy:Angel/Shreiner (Figs. 12.6, 12.7)

- The core problem being solved in ray tracing is the determination of the **closest intersection between a projector and a set of objects in the environment** (which is the visible surface problem).

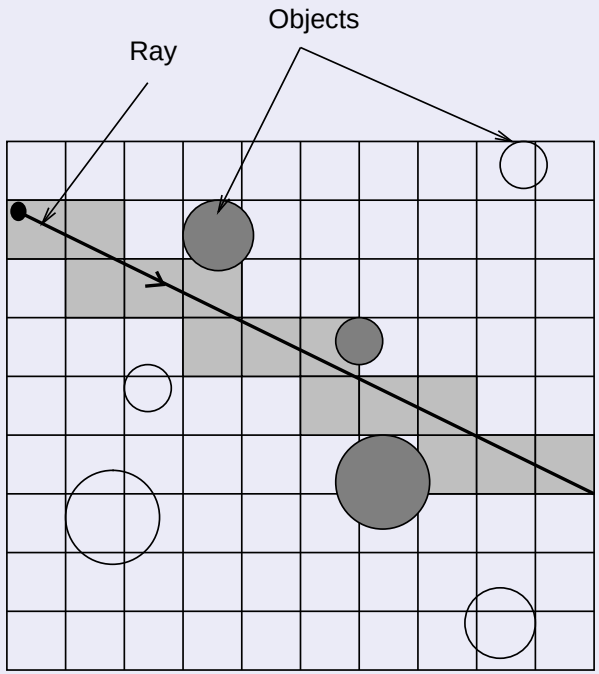
Accelerating Ray Tracing Algorithms

Two main approaches to reducing intersection calculations:

- Using bounding volumes:
  - bounding volumes are cheaper to intersect compared to object within.
- Using spatial subdivision
  - only visit objects in the vicinity of the ray.

Accelerating Ray Tracing Algorithms

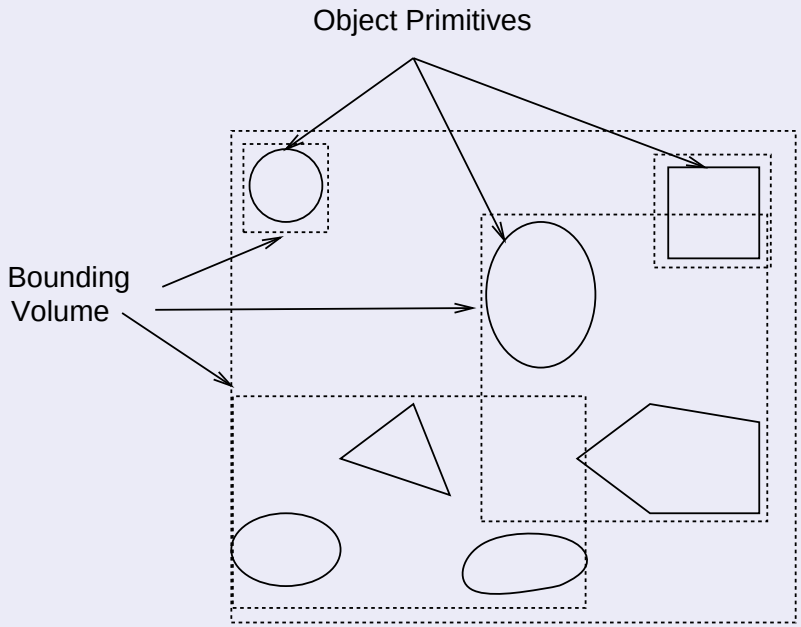
Uniform Subdivision



- Extension of line drawing algorithm to trace rays through uniformly subdivided 3D spaces.
- Fast; only objects along ray path are tested for intersection
- Uniform distribution of objects assumed, else performance is poor.

Accelerating Ray Tracing Algorithms

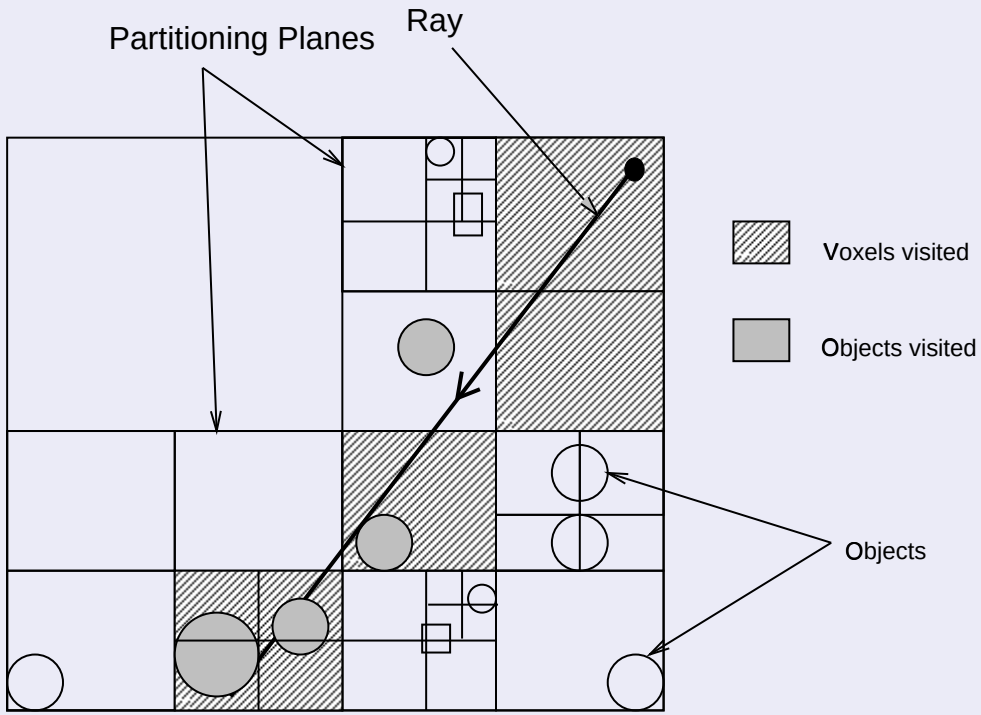
Bounding Volumes



- Enclosing objects by simple bounding volumes (spheres, parallelepipeds) permits a less expensive intersection test.
- Only if bounding volume is pierced by a ray should the object be tested.
- Idea can be extended to bounding volume hierarchies, greatly improving efficiency; whole subtrees of objects can be culled from consideration.

Accelerating Ray Tracing Algorithms

Octrees



- Very similar to a BSP tree in terms of traversal, but deal with only axis-aligned planes.
- Intersections with axis-aligned planes are less expensive.
- Fig. shows a quadtree to illustrate the idea, must use octrees in 3D.